Rate Limiting and Abuse Prevention

rate limits buy time; MFA removes the prize

Watch the 5-min video

In one line: defend in layers (edge DDoS protection and WAF, rate limits per IP, account and key, bot detection with challenges only for risky traffic), and remove the value of stolen passwords with breached-password checks, MFA and passkeys.

1 Credential stuffing

The login page gets 50,000 attempts an hour from 10,000 IP addresses, each trying a different email and password. The pairs come from other sites' breaches; people reuse passwords, so a small share succeed. Per-IP limits barely notice (five tries per IP) and per-account lockouts never fire (one try per account).

Attack	Pattern	Per-IP limit	Per-account limit	What really helps
Brute force	many passwords, one account, few IPs	catches	catches	slow hashing, delays, MFA
Password spraying	one common password, many accounts	catches if few IPs	misses	breached/common-password ban, MFA
Credential stuffing	leaked pairs, each account once, many IPs	mostly misses	misses	bot detection, MFA, passkeys

2 Layers

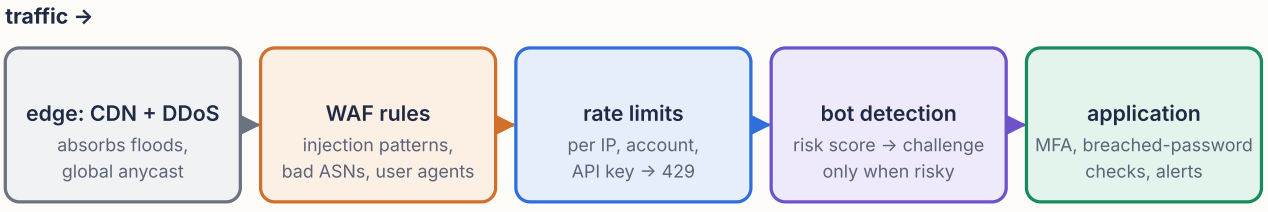


Figure 1. Abuse protection in layers, from the network edge to the application.

DDoS mitigation belongs at the edge: your servers can't absorb terabits per second. Cloudflare, AWS Shield or Azure Front Door spread traffic across a global anycast network and drop attacks before they reach you. A **web application firewall** adds managed rules (e.g. the OWASP Core Rule Set) for injection patterns, known bad user agents and suspicious networks.

3 Rate limits that work

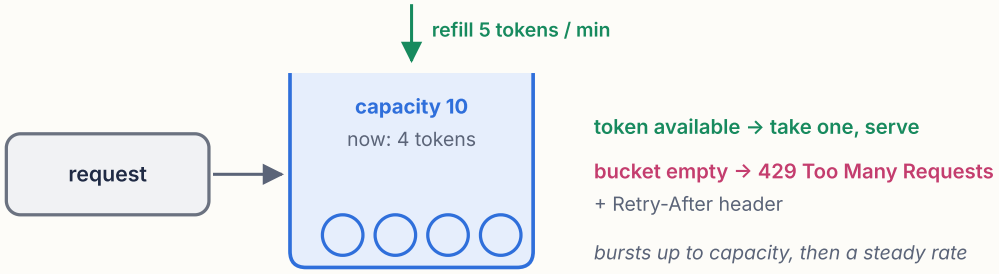


Figure 2. A token bucket allows short bursts and a steady average rate. Sliding-window counters in Redis are the common alternative.

Limit on several keys at once: **per IP** (cheap, catches simple scripts), **per account** (catches targeted guessing from many IPs), **per API key or tenant** (fairness and cost). Return **429** with `Retry-After`. For logins, prefer **progressive delays** and step-up checks over hard lockouts. NIST SP 800-63B requires limiting failed attempts per account (no more than 100 consecutive failures) and checking new passwords against breached and common-password lists.

4 Bots, CAPTCHAs and lockouts

Bot detection scores signals: request timing, TLS and browser fingerprints, JavaScript challenges, IP reputation (data-centre ranges, residential proxies). Show a CAPTCHA or managed challenge only when risk is high: CAPTCHAs annoy real users and paid solving farms beat them. **Hard lockout after five failures** lets an attacker lock out any user on purpose, a denial of service, and still misses stuffing.

5 Remove the prize

Rate limiting buys time. These take away the value of a stolen password: **MFA** stops the vast majority of stuffing logins; **breached-password checks** at sign-up and change (Have I Been Pwned's range API sends only the first 5 hex characters of a SHA-1 hash, k-anonymity); **passkeys** (WebAuthn) remove shared secrets entirely and can't be phished.

WORKED EXAMPLE: PROTECTING /LOGIN

Cloudflare sits in front with managed WAF rules and bot management. A rate-limit rule allows 5 requests per minute per IP to `/login`, then issues a challenge. Behind it, the app keeps a sliding window per account in Redis (`login:acct:{email}`) and adds a growing delay after failures. Logins from a new device need a second factor. During an attack the challenge rate spikes, most stuffing traffic never reaches the app, and an alert fires when the login failure rate jumps.

COMMON MISTAKES

- Only per-IP limits, against attackers with 10,000 residential proxy IPs.
- Hard account lockout that attackers use to lock out your users.
- Different error messages for “no such user” and “wrong password”, enabling account enumeration.
- Rate limiting /login but not password reset, sign-up or OTP verification endpoints.

6 Key terms

Credential stuffing: replaying leaked username/password pairs

Password spraying: one common password against many accounts

WAF: filters HTTP requests by attack signatures and rules

Token bucket: limiter allowing bursts up to a capacity

Progressive delay: longer wait after each failure

Passkey: WebAuthn key pair; phishing-resistant, no shared secret

7 Interview questions

Q1. How do you protect a login endpoint from credential stuffing? BASIC

Edge protection (WAF, bot management), rate limits per IP and per account, challenges only for risky traffic, progressive delays, breached-password checks, and MFA or passkeys.

Q2. Why not just lock accounts? BASIC

Attackers can trigger lockouts to deny service, and stuffing tries each account only a few times, so lockouts rarely fire.

Q3. Your API is scraped by bots using thousands of IPs. What do you do? SCENARIO

Require authentication and limit per account or API key, add bot management at the edge, cap page sizes, and watch for business-flow abuse (API6:2023).

Q4. Where should a distributed rate limiter keep its counters? SCENARIO

In a shared low-latency store such as Redis, using atomic increments or Lua scripts with TTLs; or at the edge/gateway so limits apply before your servers.

Q5. Token bucket vs fixed window: what's the difference? TRICKY

A fixed window allows double the limit across a window boundary; a token bucket (or sliding window) smooths that, allowing bursts only up to the bucket size.

Q6. Should the login form say “user not found”? TRICKY

No: different messages, or different response times, let attackers enumerate valid accounts. Return the same generic error and similar timing.

Next: SD-M-015 · Least Privilege and Zero Trust backendroadmap.com